B19-L8 lab: Build a complete test suite for a survey package

Platform: Laptop

Work in `labs/testing/` in this repository. It holds a small package, `survey_stats`, with age bands, weight helpers, a Wilson interval and an API client. You need the `dev` dependency group, which includes pytest 9, pytest-cov and statsmodels. Each step uses one lesson of module B19.

1. From the repository root, run `pip install -e . --group dev`, then `cd labs/testing` and `pytest`. One example test passes. Read every file in `src/survey_stats/`.
2. In `tests/test_bands.py`, write a parametrized test for `age_band` with named rows on both sides of 35 and 55, and a missing age (lesson B19-L3).
3. In `tests/conftest.py`, write a `make_respondents` factory fixture. Use it in `tests/test_weights.py` to test `cap_weights` with `assert_frame_equal`, including a missing weight and a check that the input is unchanged (lessons B19-L2 and B19-L4).
4. Test both errors of `normalise` and of `cap_weights` with `pytest.raises` and `match` (lesson B19-L1).
5. In `tests/test_intervals.py`, compare `wilson_interval` with statsmodels, then write a seeded coverage simulation with a tolerance of four standard errors. Mark it `@pytest.mark.slow` (lessons B19-L5 and B19-L8).
6. In `tests/test_api.py`, test `wave_count` with a fake `get`: one test for a 200 response that also checks the URL and timeout, one for a 503 (lesson B19-L7).
7. Run `pytest --cov=survey_stats --cov-branch --cov-report=term-missing`. Add tests until the Missing column is empty. Then run `pytest -m "not slow"` and check the simulation is deselected.

8. Break each function once, for example change `<` to `<=` in `age_band`. Confirm a test fails each time, then undo the change. Finally run `pytest reference_tests` and compare the reference suite with yours.

Notes